SPINLOCKS

The Problem with atomic instructions is can only work with CPU word and double word size. Cannot work with shared data structutes of custom size.

Atomic operations protect **one variable**, whereas Spinlocks protect an entire **critical section**.

1. Introduction

A Spinlock is a kernel synchronization mechanism used to protect shared data and critical sections from concurrent access by multiple CPUs.

Only one CPU can hold a spinlock at a time. If another CPU attempts to acquire the same lock, it continuously checks (spins) until the lock becomes available.

Spinlocks are primarily used in SMP (Symmetric Multiprocessing) systems.

2. Why Do We Need Spinlocks?

Atomic operations can protect only a single variable.

Example:

atomic_inc(&counter);

However, complex operations involving multiple statements or shared data structures cannot be protected using atomic operators.

Example:

remove_node(list);

update_counter();

modify_buffer();

Such code requires protection of the entire critical section.

Spinlocks provide this protection.

3. What is a Critical Section?

A Critical Section is a portion of code that accesses shared resources and must not be executed simultaneously by multiple CPUs.

Example:

spin_lock(&lock);

/* Critical Section */

shared_counter++;

shared_buffer[index] = value;

spin_unlock(&lock);

Only one CPU can execute this section at a time.

4. How a Spinlock Works

Suppose CPU0 acquires a lock:

CPU0 --> spin_lock()

CPU1 attempts to acquire the same lock:

CPU1 --> spin_lock()

Since the lock is already held:

CPU1

↓

Checks lock

↓

Checks lock

↓

Checks lock

↓

Waits until lock is released

This continuous checking is called:

Busy Waiting (Spinning)

Hence the name:

Spin Lock

5. Spinlock Workflow

Acquire Lock

↓

Enter Critical Section

↓

Access Shared Resource

↓

Release Lock

Implementation:

```
spin_lock(&lock);

/* Critical Section */

spin_unlock(&lock);
```

6. Characteristics of Spinlocks

Mutual Exclusion

Only one CPU can access protected data at a time.

Busy Waiting

Waiting CPUs continuously check the lock status.

No Context Switch

The waiting task remains active and does not sleep.

Fast

Suitable for short critical sections.

SMP-Oriented

Designed mainly for multicore systems.

7. Rules for Using Spinlocks

Rule 1: Never Sleep While Holding a Spinlock

Incorrect:

```
spin_lock(&lock);

msleep(100);
```

spin_unlock(&lock);

Reason:

The task sleeps while holding the lock.

Other CPUs wait forever.

This can lead to deadlock.

Rule 2: Keep Critical Sections Short

Correct:

spin_lock(&lock);

counter++;

spin_unlock(&lock);

Incorrect:

spin_lock(&lock);

copy_large_file();

spin_unlock(&lock);

Long critical sections waste CPU time because waiting CPUs continue spinning.

Rule 3: Avoid Blocking Functions

Never call:

msleep()

schedule()

mutex_lock()

down()

copy_to_user()

copy_from_user()

inside a spinlock protected region.

These functions may sleep or block execution.

8. Header File

#include <linux/spinlock.h>

9. Declaring a Spinlock

spinlock_t mylock;

10. Initializing a Spinlock

Static Initialization

DEFINE_SPINLOCK(mylock);

Preferred method.

Dynamic Initialization

spinlock_t mylock;

spin_lock_init(&mylock);

11. Acquiring a Spinlock

spin_lock(&mylock);

If the lock is unavailable, the CPU spins until it becomes available.

12. Releasing a Spinlock

spin_unlock(&mylock);

The lock becomes available to other CPUs.

13. Basic Example

DEFINE_SPINLOCK(mylock);

spin_lock(&mylock);

shared_counter++;

spin_unlock(&mylock);

14. Interrupt Deadlock Problem

Consider the following scenario:

Process Context:

spin_lock(&mylock);

/* Critical Section */

Before releasing the lock:

Hardware Interrupt Occurs

Interrupt Handler executes:

spin_lock(&mylock);

Problem:

ISR waits for lock

Lock owner = interrupted process

Interrupted process cannot continue

Deadlock

15. Interrupt-Safe Spinlocks

To avoid interrupt deadlocks, Linux provides:

spin_lock_irqsave()

spin_unlock_irqrestore()

These APIs disable local interrupts before acquiring the lock.

16. spin_lock_irqsave()

Syntax:

unsigned long flags;

spin_lock_irqsave(&mylock, flags);

Operations:

1. Save current interrupt state

2. Disable local interrupts

3. Acquire lock

17. spin_unlock_irqrestore()

Syntax:

spin_unlock_irqrestore(&mylock, flags);

Operations:

1. Release lock

2. Restore previous interrupt state

18. Example

unsigned long flags;

spin_lock_irqsave(&mylock, flags);

/* Critical Section */

spin_unlock_irqrestore(&mylock, flags);

19. When to Use spin_lock()

Use when shared data is accessed only by process context.

Example:

Kernel Thread ↔ Kernel Thread

20. When to Use spin_lock_irqsave()

Use when shared data is accessed by:

Process Context

+

Interrupt Handler

This prevents interrupt-related deadlocks.

21. Spinlock vs Atomic Operations

Atomic Operations Spinlocks

Protect one variable	Protect critical section
Very fast	Slightly slower
No lock required	Lock/unlock required
Counters, Flags	Lists, Queues, Buffers

22. Advantages

- Prevents race conditions.
- Suitable for SMP systems.
- Very low overhead.
- No context switching.
- Fast synchronization mechanism.

23. Disadvantages

- Wastes CPU cycles while spinning.
- Not suitable for long critical sections.
- Sleeping while holding a lock causes deadlocks.
- Poor choice for operations involving I/O or waiting.

24. Real Kernel Usage

Spinlocks are widely used for protecting:

- Linked Lists
- Queues
- Circular Buffers
- Device Structures
- Interrupt Shared Data
- Scheduler Data Structures

25. Key Takeaways

- Spinlocks protect critical sections.
- Only one CPU can hold a spinlock at a time.
- Waiting CPUs continuously spin.
- Never sleep while holding a spinlock.
- Keep critical sections short.
- Use spin_lock() for process context.
- Use spin_lock_irqsave() when interrupts can access the same resource.
- Spinlocks are best suited for short-duration synchronization on SMP systems.

What is a Spinlock?

A spinlock is a synchronization mechanism that allows only one CPU at a time to access a critical section while other CPUs wait by continuously checking the lock.

Why is it called a Spinlock?

Because waiting CPUs continuously spin (busy-wait) until the lock becomes available.

Can a task sleep while holding a spinlock?

No. Sleeping while holding a spinlock can lead to deadlocks.

Why are spinlocks suitable only for short critical sections?

Because waiting CPUs consume CPU cycles while spinning.

Difference between spin_lock() and spin_lock_irqsave()?

- spin_lock() only acquires the lock.
- spin_lock_irqsave() disables local interrupts and acquires the lock.
- spin_unlock_irqrestore() restores the interrupt state and releases the lock.

```
Skipping BTF generation for /home/sethuraj/SYNCHRONIZATION/SPINLOCK/spinlock_buffer.ko due to una
lity of vmlinux
make[1]: Leaving directory '/usr/src/linux-headers-6.8.0-124-generic'
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/SPINLOCK$ ls
Makefile       Module.symvers  spinlock_buffer.c   spinlock_buffer.mod    spinlock_buffer.mod.o
modules.order  README.md       spinlock_buffer.ko  spinlock_buffer.mod.c  spinlock_buffer.o
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/SPINLOCK$ sudo insmod spinlock_buffer.ko
[sudo] password for sethuraj:
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/SPINLOCK$ lsmod | grep spinlock_buffer
spinlock_buffer        12288  0
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/SPINLOCK$ sudo dmesg | tail -12
[523515.764867] ====================================
[530785.457687] Spinlock Buffer Demo Loaded
[530785.457727] Thread-1 -> Added 1, index=1
[530785.457767] Thread-2 -> Added 2, index=2
[530785.962783] Thread-2 -> Added 2, index=3
[530785.962891] Thread-1 -> Added 1, index=4
[530786.466783] Thread-2 -> Added 2, index=5
[530786.466789] Thread-1 -> Added 1, index=6
[530786.970789] Thread-1 -> Added 1, index=7
[530786.970795] Thread-2 -> Added 2, index=8
[530787.474832] Thread-1 -> Added 1, index=9
[530787.474841] Thread-2 -> Added 2, index=10
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/SPINLOCK$ sudo rmmod spinlock_buffer
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/SPINLOCK$ sudo dmesg | tail -12
                Final Buffer Contents:
[530871.139796] buffer[0] = 1
[530871.139797] buffer[1] = 2
[530871.139798] buffer[2] = 2
[530871.139799] buffer[3] = 1
[530871.139799] buffer[4] = 2
[530871.139800] buffer[5] = 1
[530871.139800] buffer[6] = 1
[530871.139801] buffer[7] = 2
[530871.139801] buffer[8] = 1
[530871.139802] buffer[9] = 2
[530871.139802] Spinlock Demo Unloaded
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/SPINLOCK$ _
```

What is spin_trylock()?

spin_trylock() attempts to acquire a spinlock **without waiting**.

Unlike spin_lock(), it does **not spin** if the lock is already hel

spin_trylock()

spin_trylock(&lock);

Behavior:

Lock Free
↓
Acquire Lock

Return 1

Lock Busy
↓
Do Not Wait
Return 0

spin_lock()	**spin_trylock()**
Waits for lock	Does not wait
Busy spins	Returns immediately
Guaranteed lock acquisition	May fail
Blocks execution until lock available	Non-blocking
Returns void	Returns int

What is the difference between spin_lock() and spin_trylock()?

- spin_lock() continuously spins until the lock becomes available.
- spin_trylock() attempts to acquire the lock once and immediately returns.
- It returns 1 if successful and 0 if the lock is already held.

Q: Can spinlocks be used when a resource is shared between process context and interrupt context?

Yes. Spinlocks are commonly used for this purpose. However, to avoid deadlocks caused by interrupts acquiring a lock already held by the interrupted process, the process context should use spin_lock_irqsave() and spin_unlock_irqrestore(), which disable local interrupts while holding the lock.

Shared Between	**Lock Type**
Process Context ↔ Process Context	spin_lock()
Interrupt Context ↔ Interrupt Context	spin_lock()
Process Context ↔ Interrupt Context	spin_lock_irqsave() / spin_unlock_irqrestore()

Can I use a spinlock when the resource is shared between Process Context and Interrupt Context?

Yes, but you must use an interrupt-safe spinlock.

Problem

Suppose your driver code is running in **process context** and acquires a spinlock:

spin_lock(&my_lock);

/* Critical Section */

Before releasing the lock, a hardware interrupt occurs on the **same CPU**.

The interrupt handler executes and tries to acquire the same lock:

spin_lock(&my_lock);

What happens?

Process Context acquires lock
↓
Interrupt occurs
↓
ISR tries to acquire same lock
↓
ISR spins forever

The interrupt handler cannot acquire the lock because it is already held by the interrupted process.

The interrupted process cannot run to release the lock because the ISR is currently executing.

Result

Deadlock

Solution

Disable local interrupts before acquiring the spinlock and restore them after releasing it.

Linux provides:

spin_lock_irqsave()
spin_unlock_irqrestore()

These APIs:

1. Save the current interrupt state.
2. Disable local interrupts.
3. Acquire the spinlock.
4. Execute the critical section.
5. Release the spinlock.
6. Restore the previous interrupt state.

Is kernel preemption disabled when a spinlock is acquired?

> **Yes.** Whenever a kernel thread acquires a spinlock, **kernel preemption is automatically disabled on that CPU**.

. This ensures that the task holding the spinlock is not preempted before releasing it. When the spinlock is released, preemption is re-enabled. However, normal spin_lock() does not disable interrupts; for that, spin_lock_irqsave() must be used.

Difference Between Preemption and Interrupts

spin_lock()

spin_lock(&lock);

- Disables kernel preemption
- Does NOT disable interrupts

spin_lock_irqsave()

spin_lock_irqsave(&lock, flags);

- Disables kernel preemption
- Disables local interrupts
- Saves interrupt state

Why can't we call msleep() inside a spinlock?

> msleep() puts the current task to sleep and invokes the scheduler. A spinlock disables preemption and is intended for short non-sleeping critical sections. Sleeping while holding a spinlock prevents other CPUs from acquiring the lock, causing deadlocks, hangs, or "scheduling while atomic" kernel warnings.